A Comprehensive Synthesis of LeetCode Patterns

By VIKAS V

1.1 Introduction: Deconstructing "Topics" vs. "Patterns"

A foundational challenge in algorithmic problem-solving is establishing an effective taxonomy. Official LeetCode "topics" (e.g., "Array," "String," "Hash Table") are data structures, not problem-solving techniques. A "topic" in this sense describes the tools, not the blueprint.

1.2 Master Pattern Summary Table

This table provides a high-level map of the entire LeetCode pattern ecosystem, integrating the 16 topics and 93 patterns identified in your document.[94]

Algorithmic Topic	Identified Patterns	Core Identification Key
Binary Search	**10 Patterns**	**Input is a *sorted* array, matrix, or a range of possible *answers*.[6]**
Two Pointers	**6 Patterns**	**Problem involves a sorted array, palindromes, or comparing elements from ends.**
Sliding Window	**7 Patterns**	**Problem asks for properties (max, min, count) of a *contiguous* subarray.**
Dynamic Programming	**10 Patterns**	**Problem asks for max/min, solvable by breaking into overlapping subproblems.**
DFS (Depth First Search)	**5 Patterns**	**Problem involves tree/graph traversal, exploring one path deeply.**
BFS (Breadth First Search)	**5 Patterns**	**Problem involves tree/graph traversal, exploring level-by-level (shortest path).**
Backtracking	**7 Patterns**	**Problem asks to find *all* possible solutions (subsets, permutations, combinations).[6]**
Graph Patterns	**5 Patterns**	**Problem is represented as nodes and edges (connectivity, shortest paths).[6]**
Tree Patterns	**7 Patterns**	**Problem involves traversal, path-finding, or property calculation on a tree.[6]**

Heap / Priority Queue	**4 Patterns**	**Problem asks for "Top K" elements, a median, or merging K sorted lists.[11]**
Monotonic Stack/Queue	**4 Patterns**	**Problem involves "next greater/smaller element" scenarios.**
Intervals / Merge	**4 Patterns**	**Input is a list of [start, end] pairs, asking for overlaps.[12]**
Greedy	**4 Patterns**	**Problem can be solved by making the "obvious" local choice at each step.[14]**
Linked List	**5 Patterns**	**Problem involves in-place manipulation, cycle detection, or reversal of nodes.**
Trie (Prefix Tree)	**4 Patterns**	**Problem involves string prefixes, autocompletion, or dictionary search.[16]**
Bit Manipulation	**5 Patterns**	**Problem involves efficient, low-level operations using $&$, $$, $\^$, $<<$, $>>$.[17]**

Part 2: The Comprehensive Pattern Encyclopedia

This section provides a detailed breakdown of each of the 16 topics and their 93 constituent patterns, integrating problem numbers from your document.[94]

2.1 Topic 1: Binary Search (10 Patterns)

This is a critical topic that exemplifies the need for sub-pattern analysis. The algorithm divides the search space in half repeatedly, achieving $O(\log n)$ time complexity.[94] It is applicable to sorted arrays, matrices, or even a range of possible *answers*.[8]

1. Classic Binary Search: The standard template to find an exact target in a sorted array.[8] ○ Problems: 704, 35, 278, 374.[94]
2. Search in Rotated Array: An application where the array is sorted but "rotated." An extra check determines which half is sorted.[8] ○ Problems: 33, 81, 153, 154.[94]
3. Find First/Last Position: Uses a "Lower Bound" (bisect_left) and "Upper Bound" (bisect_right) template to find the first and last occurrences of an element.[8] ○ Problems: 34, 278, 162.[94]
4. Search on Answer: The most advanced template. The search is not on the input, but on a range of possible *answers* (e.g., "Minimize the Maximum").[8] ○ Problems: 410, 875, 1011, 1283.[94]
5. Matrix Binary Search: A sorted $m \times n$ matrix is "flattened" virtually and treated as a 1D sorted array.[19] ○ Problems: 74, 240, 378.[94]
6. Peak Element: A variation to find any local maximum, where the search space is narrowed based on the comparison with neighbors. ○ Problems: 162, 852, 1095.[94]
7. Minimum in Rotated Array: A variation of the rotated array search (Pattern 2) to find the "pivot" or minimum element.
    - Problems: 153, 154.[94]
8. Koko Eating Bananas Pattern: A classic example of "Search on Answer" (Pattern 4).[8]
    - Problems: 875, 1482.[94]
9. Capacity to Ship Packages: Another "Search on Answer" (Pattern 4) problem. ○ Problems: 1011, 774.[94]
10. Split Array Largest Sum: A third "Search on Answer" (Pattern 4) problem, often considered a canonical example.
    - Problems: 410, 1231.[94]

2.2 Topic 2: Two Pointers (6 Patterns)

This technique is a fundamental pattern, primarily used on arrays or linked lists, to optimize a naive $O(n^2)$ solution to a linear $O(n)$ solution by using two pointers moving in the same or opposite direction.[94]

1. Opposite Direction (Meet in Middle): One pointer starts at the beginning (left = 0) and the other at the end (right = len(arr) - 1). They move towards each other. Classic for finding pairs in a *sorted* array.[6] ○ Problems: 1, 15, 16, 18, 167, 11.[94]
2. Same Direction (Fast & Slow - Array): One pointer (slow) maintains the "write" position, while the fast pointer scouts ahead, "reading" new elements.[25] Used for in-place modifications. ○ Problems: 26, 27, 80, 283, 844.[94]
3. Fast & Slow Pointers (Cycle Detection): Also known as Floyd's Tortoise and Hare algorithm, used in linked lists. slow moves one step, fast moves two. If they meet, there is a cycle.[3] ○ Problems: 141, 142, 202, 234, 876.[94]
4. Three Sum Pattern: A common variation of "Meet in Middle" (Pattern 1), where an outer loop fixes one element, and two pointers find the other two. ○ Problems: 15, 16, 18, 259.[94]
5. Trapping Rain Water: A specialized two-pointer problem (can also be solved with DP or stack) where pointers from both ends track the maximum height seen from that side. ○ Problems: 42, 11.[94]
6. Palindrome Check: A "Meet in Middle" (Pattern 1) variant where pointers converge from both ends, checking if s[left] == s[right].
    - Problems: 125, 680, 5.[94]

2.3 Topic 3: Sliding Window (7 Patterns)

The Sliding Window pattern is used to efficiently solve problems involving *contiguous* subarrays or substrings.[94] It optimizes naive approaches to $O(n)$ by maintaining a "window" and intelligently expanding or shrinking it.[30]

1. Fixed Size Window: The window size k is given. The window slides one element at a time, and the calculation is optimized by adding the new element and subtracting the old element.[31] ○ Problems: 643, 1456, 1343.[94]
2. Variable Size Window: The window size is not fixed. The right pointer expands, and the left pointer shrinks when a condition is met. Used to find the *shortest* or *longest* window.[34] ○ Problems: 3, 159, 340, 424, 1004.[94]
3. Longest Substring Pattern: A common "Variable Size Window" (Pattern 2) where an auxiliary data structure (hash map or set) tracks characters *in* the window.[36] ○ Problems: 3, 159, 340, 424.[94]
4. Minimum Window: A "Variable Size Window" (Pattern 2) template used to find the *shortest* window that satisfies a condition (e.g., contains all required characters). ○ Problems: 76, 209, 862.[94]
5. Anagram/Permutation Pattern: A fixed-size window where a hash map (or array) is used to track character counts to see if the window is a permutation of a target string. ○ Problems: 438, 567.[94]
6. At Most K Pattern: A "Variable Size Window" (Pattern 2) where the condition is "at most k distinct elements." ○ Problems: 340, 904, 992, 1004.[94]
7. Subarray Sum Pattern: A variable window (or prefix sum) pattern to find subarrays that sum to a target.
    - Problems: 209, 560, 930.[94]

2.4 Topic 4: Dynamic Programming (10 Patterns)

Dynamic Programming (DP) is a technique for solving complex problems by breaking them down into simpler, *overlapping subproblems* and storing their solutions.[94] The split into "1-D" and "2-D" is a highly effective way to build intuition.[5]

1. Linear DP (1D): The state $dp[i]$ depends on a fixed number of preceding states (e.g., $dp[i-1]$, $dp[i-2]$). This includes "Fibonacci-style" [40] and "Decision Making" (e.g., rob/don't rob).[41] ○ Problems: 70, 746, 198, 213, 91.[94]
2. Grid DP (2D): The state $dp[i][j]$ represents the solution for cell (i, j) in a grid, typically depending on $dp[i-1][j]$ and $dp[i][j-1]$.[44] ○ Problems: 62, 63, 64, 221, 1277.[94]
3. Knapsack (0/1): The state $dp[i][j]$ represents the max value using the first i items with a capacity j. The choice is to *include* item i or *exclude* it.[46] ○ Problems: 416, 494, 474, 1049.[94]
4. Unbounded Knapsack: A variation of Knapsack where items can be used an unlimited number of times. The transition for $dp[i][j]$ can also check $dp[i][j - \text{weight}[i]]$. ○ Problems: 322, 518, 377, 139.[94]
5. Longest Common Subsequence (LCS): The state $dp[i][j]$ is the length of the LCS between $text1[0...i]$ and $text2[0...j]$. Foundation for all string editing problems.[41] ○ Problems: 1143, 583, 712, 72.[94]
6. Longest Increasing Subsequence (LIS): A classic $O(n^2)$ DP where $dp[i]$ is the length of the LIS *ending at index i*.[46] ○ Problems: 300, 673, 354, 646.[94]
7. Partition DP: Problems involving partitioning a sequence, where $dp[i][j]$ might represent the solution for the subarray $s[i...j]$. ○ Problems: 132, 1278, 1335.[94]
8. Matrix Chain Multiplication: A DP pattern where the optimal way to multiply a chain of matrices is found by deciding the "last" multiplication.
    - Problems: 312, 1039, 1130.[94]
9. String DP: A general category that includes Palindrome DP, where $dp[i][j]$ is true if $s[i...j]$ is a palindrome.[1] ○ Problems: 5, 647, 516, 1216.[94]
10. Tree DP: DP state $dp[node]$ represents the solution for the subtree rooted at node, typically solved with a post-order traversal.
    - Problems: 337, 124, 543, 968.[94]

2.5 Topic 5: DFS (Depth First Search) (5 Patterns)

DFS is an algorithm for traversing a tree or graph by exploring as far as possible down one branch before backtracking. It uses recursion or an explicit stack.[94]

1. Tree DFS: The standard recursive traversal (pre-order, in-order, or post-order) to find paths, calculate properties, or validate structure.[3] ○ Problems: 104, 111, 112, 113, 257.[94]
2. Graph DFS: Used for finding connected components, counting islands, and detecting cycles. A visited set is required.[6] ○ Problems: 200, 695, 733, 79, 130.[94]
3. All Paths: A backtracking-style DFS used to find all possible root-to-leaf paths. ○ Problems: 257, 988, 113.[94]
4. Path Sum: A specific Tree DFS (Pattern 1) where a sum is tracked along a root-toleaf path.[1] ○ Problems: 112, 113, 437, 666.[94]
5. Matrix DFS: Applying DFS on a grid, where neighbors are adjacent cells (up, down, left, right). Used for "Flood Fill" and "Number of Islands." ○ Problems: 200, 695, 733, 79, 329.[94]

2.6 Topic 6: BFS (Breadth First Search) (5 Patterns)

BFS is an algorithm for traversing a tree or graph by exploring neighbors level by level. It uses a queue and is the go-to for finding the shortest path in an *unweighted* graph.[94]

1. Tree Level Order: The canonical BFS pattern. A queue (in Python, collections.deque) is used to traverse the tree level by level.[1] ○ Problems: 102, 103, 107, 199, 637.[94]
2. Graph BFS: Used for finding shortest paths in unweighted graphs and exploring connectivity. ○ Problems: 127, 433, 773, 1091.[94]
3. Shortest Path: Any problem asking for the "shortest," "minimum," or "fewest" steps in an unweighted graph or grid. ○ Problems: 111, 542, 994, 1162.[94]
4. Multi-source BFS: An advanced BFS where *all* source nodes (e.g., all "rotten" oranges) are added to the queue *at the beginning* to find the shortest distance from *any* source. ○ Problems: 542, 994, 317.[94]
5. Matrix BFS: Applying BFS on a grid to find the shortest path from a start cell to an end cell.
    - Problems: 286, 542, 994, 1091.[94]

2.7 Topic 7: Backtracking (7 Patterns)

Backtracking is a recursive technique for finding *all* solutions by incrementally building candidates and abandoning ("backtracking") a path as soon as it's invalid.[94] A single template can solve most of these.[3]

1. Permutations: The decision at each step is to choose one of the *remaining* available elements. A visited set is used.[3] ○ Problems: 46, 47, 31, 60.[94]
2. Combinations: The decision is to "include" or "not include" an element. An index parameter is passed to prevent duplicate combinations.[1] ○ Problems: 77, 39, 40, 216.[94]
3. Subsets (Power Set): The decision at each step is to "include" or "not include" the current element. The path is added to the results at every step.[3] ○ Problems: 78, 90.[94]
4. Partition: Problems that ask to partition a string into valid parts (e.g., palindromes). ○ Problems: 131, 93.[94]
5. Word Search: A backtracking DFS on a grid (Matrix DFS) where the path must form a target word.[3] ○ Problems: 79, 212.[94]
6. N-Queens: A classic backtracking problem with complex constraints (rows, columns, and diagonals).
    - Problems: 51, 52.[94]
7. Sudoku: A backtracking problem where the decision is to place a number (1-9) in an empty cell, followed by validating the constraints.
    - Problems: 36, 37.[94]

2.8 Topic 8: Graph Patterns (5 Patterns)

This topic covers foundational graph algorithms for problems modeled as nodes and edges.[6]

1. Topological Sort: Used for problems with *dependencies* or *prerequisites* in a Directed Acyclic Graph (DAG).[3] Solved with Kahn's Algorithm (BFS) [66] or DFS. ○ Problems: 207, 210, 269, 310.[94]
2. Union-Find (Disjoint Set Union): Used for connectivity problems in *undirected* graphs. Highly efficient for merging components and checking for cycles.[67] ○ Problems: 200, 547, 684, 721, 990, 1135.[94]
3. Shortest Path (Weighted): For *weighted* graphs, standard BFS fails. Dijkstra's algorithm is used, which is a BFS that uses a *min-priority queue*.[61] ○ Problems: 787, 743, 1334.[94]
4. Cycle Detection: Can be done with DFS (using a visiting set), BFS (Topological Sort), or Union-Find (for undirected graphs). ○ Problems: 207, 684, 685.[94]
5. Connected Components: Finding groups of connected nodes, typically solved with BFS, DFS, or Union-Find.
    - Problems: 200, 547, 323.[94]

2.9 Topic 9: Tree Patterns (7 Patterns)

This topic covers general tree operations and traversals, distinct from the specific DFS/BFS algorithms.[3]

1. Tree Traversal (Inorder): (Left, Root, Right). Used for BST validation, as it visits nodes in sorted order.[10] ○ Problems: 94, 230, 98, 173.[94]
2. Tree Traversal (Preorder): (Root, Left, Right). Used for pathfinding and serialization.[3] ○ Problems: 144, 589.[94]
3. Tree Traversal (Postorder): (Left, Right, Root). Used when a parent's solution *depends on* its children's solutions (a "bottom-up" build).[3] ○ Problems: 145, 590.[94]
4. BST Operations: A Binary Search Tree (BST) has the property left < root < right. This allows for $O(\log n)$ average-case search, insertion, and deletion. ○ Problems: 98, 235, 236, 701, 700.[94]
5. Tree Construction: Building a tree from its traversal(s) (e.g., from pre-order and inorder arrays). ○ Problems: 105, 106, 108, 889.[94]
6. Diameter/Height: Calculating tree properties, often using a "bottom-up" postorder DFS (Pattern 3).[1] ○ Problems: 104, 111, 543, 124.[94]
7. Lowest Common Ancestor (LCA): Finding the lowest shared ancestor of two nodes, typically with a recursive DFS.
    - Problems: 235, 236, 1644, 1650.[94]

2.10 Topic 10: Heap / Priority Queue (4 Patterns)

The Heap (Priority Queue) finds the min/max element in $O(1)$ time and inserts/deletes in $O(\log n)$ time. In Python, heapq is a min-heap.[11]

1. Top 'K' Elements: The most common pattern.[3] A min-heap of size k is used to find the k largest elements.[11] ○ Problems: 215, 347, 373, 658, 692.[94]
2. K-way Merge: Merging k sorted lists. A min-heap of size k stores the smallest element from each list.[3] ○ Problems: 23, 373, 378, 632.[94]
3. Median Finder (Two Heaps): A max-heap stores the "small half" and a min-heap stores the "large half," kept balanced.[3] ○ Problems: 295, 480.[94]
4. Meeting Rooms: A min-heap can be used to track the *end times* of meetings to find the minimum number of rooms required (see also 12.4).
    - Problems: 253, 1094.[94]

2.11 Topic 11: Monotonic Stack/Queue (4 Patterns)

A stack (LIFO) or queue (FIFO) that maintains a strictly increasing or decreasing order.

1. Next Greater Element: A powerful $O(n)$ technique. A *decreasing* stack is maintained. When a new element e is larger than the stack's top, e is the "Next Greater Element" for the item(s) popped.[3] ○ Problems: 496, 503, 739, 901.[94]
2. Next Smaller Element: The same logic as Pattern 1, but an *increasing* stack is maintained. ○ Problems: 84, 85.[94]
3. Sliding Window Maximum: A monotonic *decreasing deque* (double-ended queue) is used to maintain the maximum of a sliding window in $O(n)$ time. ○ Problems: 239, 1438.[94]
4. Trapping Rain Water: The Monotonic Stack (Pattern 1) is a classic way to solve this, calculating water trapped when a new, larger bar is encountered.
    - Problems: 42, 84, 85.[94]

2.12 Topic 12: Intervals / Merge (4 Patterns)

This topic involves problems where the input is a list of [start, end] intervals.[13] The *universal first step* is to *sort the intervals by their start time*.[77]

1. Merge Intervals: After sorting, iterate and merge any interval that overlaps with the previous one (current_start <= previous_end).[77] ○ Problems: 56, 57, 452, 435.[94]
2. Insert Interval: A variation where a new interval is inserted into a sorted, nonoverlapping list, followed by a merge pass. ○ Problems: 57, 715.[94]
3. Interval Intersection: A two-pointer approach for two *sorted* lists of intervals. Advance the pointer of the interval that *ends first*.[12] ○ Problems: 986, 1288.[94]
4. Meeting Rooms: Problems involving scheduling. "Can a person attend all meetings?" (sort by start, check for end-time overlap). "How many rooms are needed?" (sort by start, use min-heap) [see 10.4].
    - Problems: 252, 253.[94]

2.13 Topic 13: Greedy (4 Patterns)

Greedy algorithms rely on making the "obvious" local choice at each step, hoping it leads to a global optimum.[94]

1. Activity Selection: This is the classic "Interval Scheduling" problem. To find the *maximum* number of non-overlapping intervals, sort by *end time* and pick the next compatible interval.[82] ○ Problems: 435, 452, 253.[94]
2. Jump Game: Iterating through an array, tracking the "farthest reachable" index. This local optimum is updated at each step.[1] ○ Problems: 45, 55, 1326.[94]
3. Gas Station: A greedy problem where if the total gas is $\ge$ total cost, a solution exists. The start point is the index *after* the "dip" in the cumulative sum. ○ Problems: 134, 135.[94]
4. Two City Scheduling: A greedy problem where sorting by the *difference* in cost ($cost\_A - cost\_B$) provides the optimal solution.
    - Problems: 1029, 1196.[94]

2.14 Topic 14: Linked List (5 Patterns)

Linked List problems are fundamentally about pointer manipulation, often *in-place*.[1]

1. Reversal: The classic "reverse a linked list" algorithm using prev, curr, and next_node pointers.[3] ○ Problems: 206, 92, 25, 24.[94]
2. Fast & Slow Pointers: Used for cycle detection (see 2.3) or finding the middle node (when fast reaches the end, slow is at the middle).[1] ○ Problems: 141, 142, 876, 234.[94]
3. Merge Lists: A standard pattern to merge two *sorted* lists using a dummy head and a current pointer.
    - Problems: 21, 23, 148.[94]
4. Reorder/Remove: Using a dummy or "sentinel" head node simplifies edge cases where the head itself might be removed.[24] ○ Problems: 19, 203, 237, 143.[94]
5. Copy with Random: A complex copy that requires a hash map to map old nodes to new nodes, or a clever $O(1)$ space solution involving weaving the lists.
    - Problems: 138.[94]

2.15 Topic 15: Trie (4 Patterns)

The Trie, or Prefix Tree, is a specialized tree used to efficiently store and retrieve strings.[16] In Python, a node is implemented efficiently as a dict.

1. Implement Trie: The foundational pattern: building the Trie class with insert, search (full word), and startsWith (prefix) methods.[19] ○ Problems: 208, 211, 1804.[94]
2. Word Search: Using a Trie built from a dictionary to efficiently search for words on a grid (see 7.5), pruning the search path if no word in the trie has the current prefix. ○ Problems: 212, 79.[94]
3. Prefix/Suffix: Using the Trie's startsWith logic to find common prefixes.
    - Problems: 14, 720, 1268.[94]
4. Auto-complete: An extension of startsWith that also returns the words (or top k words) that match the prefix.
    - Problems: 642, 1268.[94]

2.16 Topic 16: Bit Manipulation (5 Patterns)

This topic is a "bag of tricks" [1] using bitwise operators ($&, |, \^, \sim, <<, >>$) for highly efficient, low-level solutions.[17]

1. Single Number: Uses the XOR ($\^$) operator's properties: $x \^ x = 0$ and $x \^ 0 = x$. All pairs cancel out, leaving the unique number. ○ Problems: 136, 137, 260.[94]
2. Counting Bits: Using bitmasks and shifts to manipulate individual bits. To *get* the i-th bit: (n >> i) & 1.[17]
    - Problems: 191, 338, 461.[94]
3. Power of Two: A number n is a power of two if $n > 0$ and (n & (n - 1)) == 0. ○ Problems: 231, 342.[94]
4. XOR Operations: A general category applying the XOR properties from Pattern 1. ○ Problems: 136, 268, 287.[94]
5. Bit Masks: Using an integer as a compact set or visited array. The i-th bit represents the presence/absence of element i.
    - Problems: 78, 1178, 1239.[94]

Part 3: Conclusion and Recommendations

This comprehensive report has successfully synthesized two extensive bodies of research into a single, unified encyclopedia. By adopting the 16-topic framework and 93pattern breakdown from your document [94], we have created a highly granular and actionable guide for mastering algorithmic problem-solving.

The key to success is to shift one's mindset from "How do I solve problem 704?" to "Which of the 10 Binary Search patterns does this problem match?".[94] This pattern-based approach [2] transforms the task from one of rote memorization to one of structured recognition.

Key recommendations for Python practice, as highlighted in your document, include [94]:

1. Start with Fundamentals: Begin with Binary Search (10 patterns), Two Pointers (6 patterns), and Sliding Window (7 patterns).
2. Master DP: Dynamic Programming (10 patterns) is one of the most varied and critical topics.
3. Graph Algorithms: Learn both DFS (5 patterns) and BFS (5 patterns) thoroughly.
4. Pattern Recognition: After solving 1-2 representative problems per pattern, you will develop the crucial skill of identifying which pattern applies to a new problem.[94]

By focusing on these 93 patterns, their identification keys, and their canonical Python implementations, a problem-solver can develop a robust and transferrable framework capable of handling the vast majority of LeetCode-style interview questions.